UNIMED CARUARU

Departamento de Tecnologia da Informação

TESTE DE SELEÇÃO — DESENVOLVIMENTO

Versão para Candidatos
2025

Antes de começar, preencha no README do repositório:
Nome completo | Telefone de contato | E-mail

1. Apresentação

Este teste tem como objetivo avaliar suas habilidades técnicas e de raciocínio em diferentes áreas do desenvolvimento de software. Ele foi desenhado para que possamos compreender não apenas o que você sabe fazer, mas também como você pensa, argumenta e toma decisões.

O teste é composto por quatro Katas independentes, cada uma focada em um contexto diferente. Você não precisa implementar tudo de forma perfeita — valorizamos muito mais a qualidade do raciocínio e a clareza das decisões do que a completude da implementação.

Leia todas as instruções com atenção antes de começar.

2. Instruções Gerais

2.1 Stack tecnológica

Você tem liberdade para escolher a linguagem e o ecossistema que preferir. Dito isso, a stack abaixo é a utilizada no dia a dia da equipe de TI da Unimed Caruaru — e por isso é a nossa sugestão, não uma imposição:

- C# / .NET — backend e APIs
- React + TypeScript — frontend web
- Python — scripts, algoritmos, análise e processamento de dados

Candidatos que utilizarem a stack recomendada terão seus projetos diretamente comparáveis ao ambiente real de trabalho, o que pode ser um diferencial durante a avaliação. Porém, qualquer outra escolha tecnológica bem justificada no README será igualmente considerada.

2.2 Entrega

A entrega deve ser feita por meio de um repositório público no GitHub. O link deve ser enviado à empresa até o prazo informado.

Estrutura esperada do repositório:

Caminho	Descrição
`/ (raiz)`	Raiz do projeto
`README.md`	Obrigatório na raiz — veja seção 2.3
`kata-1/`	Código e artefatos do Kata 1
`kata-2/`	Código e artefatos do Kata 2
`kata-3/`	Código e artefatos do Kata 3
`kata-4/`	Código e artefatos do Kata 4

2.3 README obrigatório

O arquivo README.md na raiz do projeto deve conter obrigatoriamente:

- **Nome completo**
- **Telefone de contato**
- **E-mail**
- **Stack(s) utilizada(s) e justificativa da escolha**
- **Instruções para executar cada kata localmente**
- **Comentários livres: o que você faria diferente com mais tempo?**

2.4 O que será avaliado

Além do código produzido, o avaliador irá considerar os seguintes aspectos em cada kata:

Critério	Descrição
Clareza do raciocínio	O candidato explica suas decisões? Há lógica coerente nas escolhas?
Qualidade do código	Legibilidade, organização, boas práticas e tratamento de erros.
Decisões de engenharia	Identificação de trade-offs, limitações e alternativas consideradas.
Análise de requisitos	Capacidade de questionar, refinar e documentar o que foi pedido.
Banco de dados	Modelagem adequada e consultas eficientes quando aplicável.
Testes e qualidade	Presença de testes unitários ou estratégia de validação descrita.

3. Kata 1 — Lógica e Algoritmos

Kata 1 **Fila de Triagem**
Tema: Clínica / Healthtech | Tempo estimado: 1h – 1h30

3.1 Contexto

Uma clínica médica precisa organizar a fila de atendimento de pacientes. Cada paciente chega com um nível de urgência e um horário de chegada. O sistema deve ordenar a fila respeitando as seguintes regras de negócio:

Regras da fila de triagem:
1. Pacientes com urgência CRÍTICA sempre têm prioridade máxima.
2. Pacientes com urgência ALTA têm prioridade sobre MÉDIA e BAIXA.
3. Dentro do mesmo nível de urgência, atende-se por ordem de chegada (FIFO).
4. Pacientes idosos (60+ anos) com urgência MÉDIA sobem automaticamente para ALTA.
5. Pacientes menores de 18 anos com qualquer urgência ganham +1 nível de prioridade.

3.2 O que você deve entregar

Parte A — Implementação (obrigatório)

Implemente uma função ou módulo que receba uma lista de pacientes e retorne a fila ordenada. Cada paciente tem:

- Nome (string)
- Idade (inteiro)
- Nível de urgência: CRÍTICA, ALTA, MÉDIA ou BAIXA
- Horário de chegada (timestamp ou string HH:MM)

Escreva ao menos dois testes unitários que validem as regras de negócio, incluindo os casos de borda das regras 4 e 5.

Parte B — Análise escrita (obrigatório)

Responda no README do kata ou em um arquivo ANALISE.md dentro de kata-1/:

1. Qual estrutura de dados você escolheu para modelar a fila e por quê?
2. Qual a complexidade de tempo do seu algoritmo de ordenação? Seria diferente se a lista tivesse 1 milhão de pacientes?
3. As regras 4 e 5 interagem entre si? Descreva o que acontece quando um paciente tem 15 anos e urgência MÉDIA.
4. Se a clínica adicionasse uma 6ª regra amanhã, como seu código lidaria com essa extensão?

Parte C — Banco de dados (opcional, mas valorizado)

Modele as tabelas necessárias para persistir pacientes, filas e atendimentos. Entregue o script SQL ou a migration. Não é necessário integrar com a Parte A.

> **Dica:** Não existe uma única solução correta. O que mais nos interessa é como você estrutura o problema e documenta suas escolhas.

4. Kata 2 — Feature Full-Stack

Kata 2 **Painel de Tarefas**
Tema: Gestão / Produtividade | Tempo estimado: 2h – 3h

4.1 Contexto

Uma equipe interna precisa de uma ferramenta simples para gerenciar tarefas do dia a dia. O produto é um painel web com backend e frontend integrados. Os requisitos foram passados de forma informal pelo cliente interno — sua primeira tarefa é identificar as ambiguidades antes de implementar.

4.2 Requisitos informais (como o cliente descreveu)

> *"Preciso de uma tela onde eu veja minhas tarefas. Cada tarefa tem um título e uma situação.*
> *Quero poder criar novas, marcar como feita e deletar as que não preciso mais.*
> *Ah, e precisa ter um filtro pra eu ver só as pendentes ou só as concluídas.*
> *Se der, queria também uma prioridade nas tarefas. Mas isso pode ficar pra depois."*

4.3 O que você deve entregar

Parte A — Análise de requisitos (obrigatório)

Antes de qualquer código, escreva em um arquivo REQUISITOS.md dentro de kata-2/ respondendo:

5. Liste pelo menos 3 ambiguidades ou informações faltantes nos requisitos acima.
6. Para cada ambiguidade, escreva a pergunta que você faria ao cliente e a decisão que você tomou na ausência da resposta.
7. Defina os Requisitos Funcionais (RF) e Requisitos Não Funcionais (RNF) formalmente, com base no que você entendeu.
8. O requisito de prioridade foi marcado como "pode ficar pra depois". Como você trataria isso no seu backlog?

Parte B — Backend (obrigatório)

Implemente uma API REST com os seguintes endpoints mínimos:

- GET /tasks — listar tarefas (com suporte a filtro por status)
- POST /tasks — criar tarefa
- PATCH /tasks/{id} — atualizar status ou outros campos
- DELETE /tasks/{id} — remover tarefa

A persistência pode ser em memória, SQLite, arquivo JSON ou banco relacional — sua escolha, justificada no README. Inclua tratamento de erros e retornos HTTP adequados.

Parte C — Frontend (obrigatório)

Implemente uma interface funcional que consuma os endpoints da Parte B. A interface deve conter:

- Listagem de tarefas com indicação visual de status
- Formulário para criação de nova tarefa
- Ação para concluir e para excluir uma tarefa
- Filtro por status (pendente / concluída)

Não há exigência de design elaborado. Clareza e funcionalidade são suficientes.

Parte D — Engenharia (obrigatório)

Responda no README do kata ou em ENGENHARIA.md dentro de kata-2/:

9. Que decisões de arquitetura você tomou no backend (separação de responsabilidades, camadas, etc.)?
10. Como você garantiria que a API é confiável em produção? Mencione pelo menos dois aspectos de qualidade/observabilidade.
11. Se o sistema precisasse suportar múltiplos usuários com autenticação, o que mudaria na sua arquitetura atual?

5. Kata 3 — Análise de Engenharia de Software

Kata 3 **Sistema Legado em Colapso**
Tema: E-commerce / Logística | Tempo estimado: 1h – 1h30

5.1 Contexto

Você acaba de entrar em um time que mantém um sistema de pedidos de e-commerce. O sistema funciona há 5 anos e processa cerca de 800 pedidos por dia. Nas últimas semanas, os seguintes problemas foram relatados:

Relatório de incidentes recentes:
- O endpoint de consulta de pedidos demora 8–12 segundos em horário de pico.
- Dois pedidos foram criados em duplicidade na última semana.
- Um bug de cálculo de frete foi corrigido em produção diretamente (sem PR, sem teste).
- O código da camada de negócio tem +4.000 linhas em um único arquivo.
- Não há testes automatizados. Nenhum.

Você recebeu a tarefa de elaborar um plano de ação técnico para apresentar ao seu gestor na próxima semana. Não há sprint liberada para isso ainda — você precisa priorizar e justificar.

5.2 O que você deve entregar

Entregue um documento PLANO.md (ou .pdf) dentro de kata-3/ com as seguintes seções:

Seção 1 — Diagnóstico

Para cada um dos 5 problemas listados, identifique:

- A causa raiz mais provável
- O risco que ele representa (técnico e de negócio)
- Se é um problema urgente ou importante (use a Matriz de Eisenhower ou equivalente)

Seção 2 — Plano de ação

Proponha ações concretas para os 3 problemas que você priorizaria primeiro. Para cada ação:

- Descreva o que será feito em termos técnicos
- Estime o esforço (horas/dias, pode ser aproximado)
- Indique o critério de sucesso — como você saberá que o problema foi resolvido?

Seção 3 — Decisão de arquitetura

O arquivo de 4.000 linhas precisa ser refatorado. Você tem duas opções sendo discutidas no time:

Opção A — Refatoração incremental	Opção B — Reescrita do módulo

Extrair módulos menores gradualmente, mantendo o sistema funcionando durante o processo. Menor risco, mais lento.	Reescrever o módulo do zero com arquitetura limpa e testes. Maior risco de regressão, entrega mais rápida do resultado final.

Argumente qual das duas opções você escolheria e por quê. Considere o contexto descrito (sem testes, sistema em produção, time ocupado). Não há resposta errada — a qualidade da argumentação é o que será avaliado.

Seção 4 — Requisitos Não Funcionais ignorados

Identifique pelo menos 3 Requisitos Não Funcionais (RNFs) que claramente não estão sendo atendidos no sistema descrito. Para cada um:

- Nomeie o RNF (ex.: desempenho, disponibilidade, manutenibilidade...)
- Explique por que ele está comprometido com base nos incidentes descritos
- Proponha uma métrica mensurável para monitorá-lo

Importante: Este kata não exige nenhum código. O que será avaliado é a qualidade da análise, a clareza das decisões e a maturidade técnica demonstrada no documento.

6. Kata 4 — Análise e Transformação de Dados

Kata 4 **Pipeline de Relatório**
Tema: Financeiro / Dados | Tempo estimado: 1h30 – 2h

6.1 Contexto

Você recebeu três arquivos CSV exportados de diferentes sistemas de uma empresa fictícia de logística. Os dados estão sujos, inconsistentes e em formatos diferentes entre si. Sua tarefa é construir um pipeline de transformação que produza um relatório consolidado com indicadores de desempenho.

Os três arquivos simulados que você deve criar (ou que serão fornecidos) são:

- pedidos.csv — colunas: id_pedido, data_pedido, id_cliente, valor_total, status
- clientes.csv — colunas: id_cliente, nome, cidade, estado, data_cadastro
- entregas.csv — colunas: id_entrega, id_pedido, data_prevista, data_realizada, status_entrega

Problemas intencionais nos dados (você deve tratar todos):
• Datas em formatos mistos (DD/MM/AAAA, AAAA-MM-DD, timestamps).
• Valores monetários com vírgula como separador decimal em alguns registros.
• Campos nulos em colunas obrigatórias.
• IDs de pedido em entregas.csv que não existem em pedidos.csv (orphan records).
• Nomes de cidades com grafias inconsistentes (ex.: 'São Paulo', 'sao paulo', 'SAO PAULO').

6.2 O que você deve entregar

Parte A — Pipeline de transformação (obrigatório)

Implemente um script ou módulo que:

- Leia os três arquivos CSV
- Aplique as limpezas e normalizações necessárias
- Gere um arquivo consolidado (CSV ou JSON) com as seguintes colunas por pedido:

id_pedido | nome_cliente | cidade_normalizada | estado | valor_total | status_pedido | data_pedido | data_prevista_entrega | data_realizada_entrega | atraso_dias | status_entrega

O campo atraso_dias deve ser calculado como a diferença entre data_realizada e data_prevista (negativo = antecipado, nulo = ainda não entregue).

Parte B — Indicadores consolidados (obrigatório)

A partir do arquivo consolidado, calcule e exiba (em tela ou em um segundo arquivo):

- Total de pedidos por status
- Ticket médio por estado
- Percentual de entregas no prazo vs. com atraso
- Top 3 cidades com maior volume de pedidos
- Média de atraso em dias para pedidos com atraso

Parte C — Análise escrita (obrigatório)

Responda em um arquivo ANALISE.md dentro de kata-4/:

12. Quais foram as principais decisões de tratamento que você tomou? (ex.: o que fazer com registros órfãos, como normalizar cidades)
13. Seu pipeline é idempotente? Ou seja, rodá-lo duas vezes produz o mesmo resultado? Justifique.
14. Se esse pipeline fosse executado diariamente com arquivos de 10 milhões de linhas, o que você mudaria na abordagem?
15. Que testes você escreveria para garantir a qualidade das transformações?

Dica: Você pode criar os arquivos CSV de exemplo com dados fictícios para demonstrar o pipeline funcionando. Inclua-os no repositório.

7. Checklist Final

Antes de enviar o link do repositório, verifique:

☐	README.md na raiz com nome, telefone, e-mail, stack e instruções
☐	Kata 1: função implementada + testes unitários + análise escrita
☐	Kata 2: REQUISITOS.md + API funcionando + frontend funcional + análise de engenharia
☐	Kata 3: PLANO.md com as 4 seções preenchidas
☐	Kata 4: pipeline implementado + análise escrita
☐	Repositório público e acessível
☐	Código com commits organizados (não um único commit gigante)
☐	Link enviado por e-mail para romulo.eduardo@squarelabs.com.br com assunto correto

Boa sorte! Em caso de dúvidas sobre as instruções, entre em contato com a empresa.